UNIVERSITY OF GEORGIA / ECSE 4235 / FALL 2026

PARMCO

SOFTWARE DOCUMENTATION

Technical reference for the Raspberry Pi motor-control application, ARM32 stepper driver, Bluetooth communication service, AS5600 magnetic encoder, IR RPM sensing, CSV data logging, and SwiftUI iPhone interface.

DOCUMENT INFORMATION	DETAIL
Prepared	11 November 2026
Course and project	University of Georgia / ECSE 4235 / PARMCO
Team	Group 1
Raspberry Pi platform	Raspberry Pi 4; C11, ARM32 assembly, GLib/GIO, BlueZ
iPhone platform	SwiftUI, Foundation, Combine, CoreBluetooth
Documented source	Seven project files supplied for this software review
Document contents	Architecture, functions, library attribution, code flags, interface protocol and source traceability

Documentation basis

Descriptions are derived from the supplied project source files and the ECSE 4235 PARMCO guide. Implementation details and source-line references identify the behavior represented by these materials. Hardware performance figures are not inferred from source code.

1. Software overview and source inventory

PARMCO uses an iPhone application as the operator interface and a Raspberry Pi process as the motor-control and measurement system. The Bluetooth service forwards validated commands to the motor controller, which coordinates assembly stepping, the IR sensor, the AS5600 encoder and asynchronous CSV logging.

Project filename	Language	Primary responsibility
motor_cp3.c	C	Motor and IR workers, AS5600 reader, Manual control, position commands, CSV logging queue.
motor_cp3.h	C header	Shared declarations for initialization, shutdown, session state, command dispatch and telemetry.
parmco_ble_cp3.c	C	BlueZ D-Bus/GATT service, advertisement, command receipt, notification scheduling and safety transitions.
motorDriver.s	ARM32 assembly	H-bridge enable, phase lookup, commanded phase advancement and generic-timer wait.
PARMCOApp.swift	Swift	iOS application entry point and scene composition.
BluetoothManager.swift	Swift	Bluetooth central manager, characteristics, command writes, notifications and state publishing.
ContentView.swift	SwiftUI	Operator controls, live status, calibration/positioning, logging and input validation.

1.1 Command and measurement paths

Command path

ContentView.swift -> BluetoothManager.send() -> BLE command characteristic -> BlueZ WriteValue -> cp3_command() -> motor worker -> motorDriver.s -> E4235_Write -> 4N25 / SN754410NE / stepper motor.

Measurement and data path

GPIO17 -> E4235_Read -> IR worker; AS5600 GPIO2/GPIO3 I2C bus -> encoder worker; sensor state -> cp3_telemetry() -> BLE notifications -> BluetoothManager.receive() -> SwiftUI. Log snapshots -> GAsyncQueue -> CSV writer thread.

2. Library attribution and dependencies

Course-provided class library

E4235.h and libE4235.a are provided by the University of Georgia ECSE 4235 course, not authored by AI. Course library documentation revision 1.1.2 (12 February 2026) identifies the instructor-supported GPIO interface and course contributor credits.

Library / provider	How it works	Purpose in PARMCO
E4235 UGA ECSE 4235	E4235_Init initializes GPIO access; E4235_Select configures pin direction; E4235_Read acquires digital input; E4235_Write drives outputs.	Motor phase and enable signals use class library functions; IR pulses are sampled through E4235_Read.
BlueZ Linux Bluetooth project	Provides org.bluez GATT and LEAdvertisingManager interfaces through system D-Bus.	Registers the peripheral service, advertises its UUID and supports iPhone command/notification exchange.
GLib / GIO GNOME project	GMainLoop dispatches events; GDBus registers methods and objects; GVariant encodes messages; GThread, GMutex and GAsyncQueue coordinate concurrent operations.	Separates responsive BLE processing from motor stepping, sensor acquisition and CSV file output.
Linux i2c-dev Linux kernel userspace API	The /dev/i2c-1 device and ioctl(I2C_RDWR) perform a register-address write followed by a repeated-start read.	Reads AS5600 sensor status and angle through the Raspberry Pi I2C controller at address 0x36.

2.1 Hardware and source ownership

The project-specific C, assembly and Swift sources implement the application logic. E4235 is an externally supplied instructional library, and BlueZ, GLib/GIO, Linux kernel APIs and Apple frameworks are externally maintained dependencies. Attribution distinguishes these components from project code.

AS5600 communication uses the Linux I2C userspace interface; digital motor-output and IR-input operations use the E4235 class library. This distinction is included for technical accuracy and library attribution.

2. Library attribution and dependencies (continued)

Library / provider	How it works	Purpose in PARMCO
SwiftUI Apple	Declarative App, WindowGroup, NavigationStack, GroupBox and state-driven controls create the app interface.	Displays motor status and provides speed, direction, calibration, position and logging controls.
CoreBluetooth Apple	CBCentralManager scans and connects; CBPeripheral discovers GATT services and characteristics and executes writes/notification subscriptions.	Establishes the iPhone-to-Pi BLE channel and receives status information.
Combine Apple	ObservableObject and @Published propagate state changes; Timer.publish(...).autoconnect() produces periodic events.	Updates the UI when BLE telemetry changes; drives the app heartbeat.
Foundation Apple	Provides NSObject, Data, text conversion, numerical parsing and common Swift runtime types.	Supports BLE manager objects and UTF-8 command/response handling.
ISO C library / libm	stdio/string/stdlib and math functions support formatting, parsing, file writing, fmod, fabs, lround and limits.	Implements command validation, angular conversion, motor-rate calculations and data logging.
POSIX / Linux facilities	open, close, ioctl, signals, file descriptors, timestamps and Linux I2C data structures.	Supports sensor transactions, process shutdown and input/output operations.
GCC / GNU binutils	Compiles C11, assembles ARM32 source and links the requested libraries.	Creates the Raspberry Pi executable from project C and assembly files.

2.2 Environment and package records

The BLE source identifies Raspberry Pi OS 12 (Bookworm) and BlueZ 5.66 in its header comments. The source files do not establish the installed GLib/GIO patch version, Xcode build number or iOS SDK version. Library versions should be recorded from the actual build environment when included in a reproducibility record.

3. motor_cp3.c - motor control and sensing

3.1 Initialization, pin mapping and initial state

cp3_init() initializes E4235, selects GPIO26 as an enable output, sets the bridge to an inactive condition, configures GPIO18/23/24/25 as stepper phase outputs and GPIO17 as the IR input, and starts the motor, IR, encoder and logging worker infrastructure. The initial control state is clockwise, 20 RPM and stopped.

Source flag / location	Configured value	Operational meaning
STEP_MODE / STEPS_PER_REV motor_cp3.c L38-39	1 / 400	Half-step mode; 0.9 degrees for each commanded half-step.
ENABLE_GPIO / phase outputs L40-41; initialization L602-645	GPIO26; 18, 23, 24, 25	GPIO26 operates the 4N25 enable; the other outputs drive SN754410NE inputs.
IR_GPIO / IR_ACTIVE_LEVEL L40; L47	17; active LOW	IR worker detects active-low pulses.
MIN_RPM / NORMAL_MAX_RPM L42-43	10 / 100 RPM	Pi-side normal operating speed bounds.
TEST_MAX_RPM L44-45	400 RPM	Pi-side ceiling when over-range testing is enabled.
running / want_bridge / bridge_on L83-85	Boolean flags	Separates motor-control requests from the worker-owned physical bridge state.

3.2 Continuous rotation and IR measurement

motor_worker() reads state under a GMutex and performs the timed **motor_wait_step()** operation outside that lock. A START command sets persistent running state; speed and direction can be updated through subsequent commands. STOP or DISABLE requests bridge-off. Disconnection and process shutdown also stop the motor as safety actions.

ir_worker() samples GPIO17 using E4235_Read. The pulse-processing function applies approximately 4 ms debounce, excludes pulses closer than 55 ms, averages up to four intervals and calculates RPM using two pulses per revolution. The IR reading returns to zero when stopped or when valid pulses are absent long enough.

IR RPM = 60 x number_of_intervals / (2 x total_interval_seconds)
Nominal step rate = round(desired_RPM x 400 / 60)

3. motor_cp3.c - encoder and position control

3.3 AS5600 acquisition

The encoder worker opens **/dev/i2c-1**, addresses the AS5600 at **0x36** and obtains status (0x0B) and 12-bit RAW_ANGLE (0x0C/0x0D) using a combined I2C_RDWR transaction. A successful read with magnet-detected bit 0x20 supplies a valid angle. Weak-field bit 0x10 triggers a warning without automatically rejecting the measurement.

Sensor sampling is requested every 10 ms. A signed wrapped angular difference supports cumulative position across revolutions. Encoder-derived RPM is calculated over an approximately 0.40-second window. A position command requires a sample that is no more than 250 ms old.

```
raw_angle_deg = raw_count * 360.0 / 4096.0

enc_angle_deg = wrap360((raw_angle_deg - zero_offset) * sign)

enc_cum_deg = (raw_cumulative - calibration_base) * sign
```

3.4 Zero calibration and individual target movements

CAL stores a raw-angle zero reference in volatile memory and resets the commanded step reference. ZERO and ANGLE request movement toward a specified position. If motor-to-encoder direction has not been established, the controller performs a short commanded clockwise probe. Target moves use the shortest signed angular error and reduce speed as the target approaches.

Position-control flag	Value	Behavior
POSITION_TOLERANCE_DEG L62	1.0 degree	Target-reached tolerance.
POSITION_TIMEOUT_S L63	20 s	Maximum position-move duration.
POSITION_MAX_STEPS L64	600 steps	Limits commanded steps during a move.
POSITION_PROBE_* L68-71	10 RPM; 8-30 steps	Determines the motor/encoder direction relationship.
POSITION_PROGRESS_* L73-74	30 steps; 1.8 degrees	Aborts motion without adequate reduction in error.

A completed discrete move stops the motor and disables the bridge. Continuous idle feedback hold and automatic speed regulation are not implemented in the documented code.

3. motor_cp3.c - command handling and logging

3.5 Command dispatcher

cp3_command() validates incoming ASCII commands and returns bounded ACK or ERR text. The BLE callback queues or updates state rapidly; the motor worker performs the physical steps. A separate STATE notification reports Pi-side movement status.

Command group	Result and control rules
START; STOP; ENABLE; DISABLE; ONE	Continuous run, stop/bridge disable, bridge enable, bridge disable or one commanded revolution. Safety and busy-state rules apply.
DIR:CW; DIR:CCW; FASTER; SLOWER; RPM:n	Direction changes when stopped; Faster/Slower change the step-rate setting in small increments; RPM values are validated.
MODE:MANUAL; MODE:MAINTAIN	Manual is implemented; Maintain returns an explicit pending/not-implemented error.
SIGN:+; SIGN:-; CAL; ZERO; ANGLE:n	Encoder direction indication, in-memory zero calibration and finite angle/return moves, guarded by sensor and motor state.
UNLOCK:ON; UNLOCK:OFF	Enables over-range test commands; the unlock state resets after stopping.
LOG:ON; LOG:OFF; MARK; STATUS; PING	Starts/stops CSV recording, records a marker, requests status and acknowledges the periodic connectivity message.

3.6 Concurrent logging

LOG:ON creates a timestamped **cp3_test_*.csv** file inside the local **logs/** directory. The GLib event timer requests a row every 10 ms. Under a mutex, the code snapshots the current motor and sensor values, queues the row in a GAsyncQueue, and writes it to disk using a dedicated thread. This design keeps file I/O out of motor stepping.

```
time_s,mode,cmd_rpm,cmd_angle_deg,step_count,step_angle_deg,
enc_angle_deg,enc_cum_deg,ir_rpm,enc_rpm,app_rpm,enable,event
```

step_angle_deg represents commanded steps multiplied by 0.9 degrees; enc_cum_deg is encoder position unwrapped relative to calibration. app_rpm is the IR-derived RPM value made available to the phone, not a separate sensor. The 10 ms tick is nominal; the actual sampling rate must be evaluated from CSV timestamps.

4. motor_cp3.h - shared interface

The header protects declarations with an include guard, imports GLib and size_t definitions, and exposes seven functions shared by the motor/controller module and Bluetooth server. It contains declarations rather than hardware-control logic.

Declaration	Purpose
cp3_init() / cp3_shutdown()	Initialize GPIO and workers; request safe stop and release worker resources on shutdown.
cp3_connected() / cp3_disconnected()	Re-establish initial session state; disable the motor after a connection ends.
cp3_command(command, reply, capacity)	Dispatch Pi-side commands and return a bounded acknowledgment or error string.
cp3_telemetry(slot, out, capacity)	Produce a selected telemetry frame from current system state.
cp3_log_tick()	Request a logging sample while logging is active.

5. motorDriver.s - ARM32 motor interface

The assembly declares unified ARMv7-A syntax, a four-entry full-step table, an eight-entry half-step table and persistent phase index storage. All H-bridge output operations call **E4235_Write** through the standard ARM calling convention.

Function / source	Interface and operation
motor_enable / L37-48	r0 carries on/off; its low bit is written to GPIO26.
motor_apply_phase / L50-101	r0 selects step mode; four phase bits are written to GPIO18, 23, 24 and 25.
motor_step / L103-136	r0 is direction, r1 is mode; advances and wraps the phase index, then updates outputs.
motor_wait_step / L138-186	r0 is wait period in microseconds, r1 direction, r2 mode; ARM generic timer determines the next step deadline.

Full-step patterns: 5, 6, 10, 9 **Half-step patterns:** 5, 4, 6, 2, 10, 8, 9, 1. The .note.GNU-stack section requests a non-executable stack. The busy-wait operates in the motor thread, not in the GATT event loop.

6. parmco_ble_cp3.c - Bluetooth service

6.1 GATT services and message handling

The Raspberry Pi exports D-Bus ObjectManager, BlueZ GattService1, two GattCharacteristic1 instances and LEAdvertisement1. The program registers a GATT application, advertises the project service, and enters a GLib main loop to receive Bluetooth writes and send notifications. WriteValue accepts bounded ASCII command text, removes a trailing newline and dispatches the result to **cp3_command()**.

GATT role	UUID	Use
Primary service	E4235001-2026-4000-8000-000000000001	Advertised service identifier for discovery.
Command (write)	E4235001-2026-4000-8000-000000000002	UTF-8 text command plus newline.
Response (notify)	E4235001-2026-4000-8000-000000000003	ACK/ERR response and rotating telemetry notifications.

6.2 Telemetry and disconnection handling

A 200 ms GLib timer alternates among four telemetry formats, so each type is ordinarily refreshed approximately every 0.8 seconds while notifications are active. A separate 10 ms timer requests CSV snapshots. StartNotify enters a new phone session; StopNotify and observed BlueZ connection loss trigger safety-state handling.

Notification	Payload
RPM:<ir>,<enc>	Measured IR and encoder-derived speed.
POS:<angle>,<cal>,<present>,<sign>	Calibrated angle, calibration flag, encoder availability and orientation.
STATE:<running>,<dir>,<rpm>,<unlock>	Movement state, direction, desired speed and test unlock.
LOG:<enabled>,<seconds>,<file>	CSV logging indicator, duration and file name.
ACK:<action> / ERR:<reason>	Command acknowledgment or rejection; not independent evidence of motor movement.

7. iPhone application

7.1 PARMCOApp.swift

The **@main PARMCOApp: App** declaration starts the iOS application and supplies a **WindowGroup** containing ContentView. The entry-point file does not perform Bluetooth I/O or motor-control calculations.

7.2 BluetoothManager.swift

The BLE manager inherits from NSObject, adopts ObservableObject, and implements CBCentralManagerDelegate and CBPeripheralDelegate. It scans for the project service UUID, connects to the Pi, discovers the command/write and response/notify characteristics, subscribes to notifications and sends commands using CoreBluetooth writes with response.

Published state / method	Interface role
status, isReady, lastSent, lastReceived, lastReply	Expose connection progress, commands and acknowledgments.
actualIRRPM, encoderRPM, encoderAngle, encoderPresent, calibrated	Reflect sensor measurements, availability and calibration state.
encoderSignVerified, running, clockwise, desiredRPM, unlocked	Expose direction, movement and requested speed/test state.
logging, logSeconds, logFile	Reflect Pi-side CSV recording activity.
connect(), disconnect(), send()	Manage BLE connection and transmit commands; disconnect requests STOP first when ready.
heartbeat()	Send a quiet PING once per second while connected; the Pi acknowledges it without an inactivity-stop requirement.

7.3 ContentView.swift

The SwiftUI screen provides Bluetooth connection controls, Manual mode selection, motor speed and direction, START/STOP, IR/encoder telemetry, sign selection, zero calibration, angle targets, test unlock, CSV recording and diagnostics. It validates angle targets within 0-360 degrees and provides visible input-error messages. The Maintain option is displayed but is not available for active control.

In normal operation, the requested RPM is restricted to 10-100. The project team has confirmed that the currently deployed app permits up to 400 RPM when over-range testing is unlocked. This deployment setting is distinct from the Pi-side limit documented in **motor_cp3.c**; the submitted Swift source should be synchronized with the deployed interface for exact reproducibility.

8. Build configuration and code flags

8.1 Raspberry Pi build and launch

```
gcc -std=c11 -Wall -Wextra -O2 -pthread -no-pie \
  parmco_ble_cp3.c motor_cp3.c motorDriver.s -o parmco_cp3 \
  $(pkg-config --cflags --libs gio-2.0 glib-2.0) -lE4235 -lm
sudo ./parmco_cp3
```

Flag / element	Purpose
-std=c11	Compile the C sources in C11 language mode.
-Wall -Wextra	Enable additional GCC warnings.
-O2	Use compiler optimization while preserving C language behavior.
-pthread	Enable build/link support for multithreaded code.
-no-pie	Request a non-PIE executable for the ARM assembly-integrated build.
pkg-config --cflags --libs gio-2.0 glib-2.0	Provide GLib/GIO include and link flags from the installed packages.
-lE4235 -lm	Link the course GPIO library and mathematical functions.
sudo ./parmco_cp3	Start the built server manually with suitable device access.

8.2 Source configuration register

Flag ID / source	Setting	Function
CFG-01 motor_cp3.c L38-45	Half-step/400; 10-100/400 RPM	Motor mode, normal bounds and unlocked ceiling.
CFG-02 motor_cp3.c L47-60	IR 4 ms debounce; encoder 10 ms	Measurement filtering and nominal acquisition period.
CFG-03 motor_cp3.c L62-74	1 degree; 20 s; 600 steps	Position completion and abort protections.
CFG-04 BluetoothManager.swift L73	PING at 1-second interval	Connection-status heartbeat, not motor inactivity watchdog.
CFG-05 parmco_ble_cp3.c timers	200 ms / 10 ms	Rotating telemetry notifications and logging request cadence.

9. Code attribution and annotation flags

The course requires attribution of externally supplied libraries and clear identification of student modifications to AI-authored project code. The supplied sources contain function comments, configuration declarations and recognizable public entry points. This documentation records those source flags without assigning authorship to edits that cannot be independently established.

Source annotation audit

The supplied seven-file set does not contain paired student-edit boundary markers. The table below identifies existing project code flags and review points; it is not a claim that student-edit annotations are already present.

9.1 Existing technical code flags

Source	Existing flags and identifiers	Review purpose
motor_cp3.c	STEP_MODE, STEPS_PER_REV, TEST_MAX_RPM; POSITION_*; AS5600_*	Records actuator/sensor configuration, RPM bounds and position protections.
motor_cp3.h	MOTOR_CP3_H include guard and cp3_* declarations	Defines the public C interface and prevents multiple header expansion.
parmco_ble_cp3.c	UUID_SERVICE / UUID_COMMAND / UUID_RESPONSE; *_PATH constants	Defines GATT identifiers and D-Bus object locations.
motorDriver.s	full32, half32, phase32; .global / .type; .note.GNU-stack	Identifies phase tables, exported ABI and stack metadata.
PARMCOApp.swift	@main, App, WindowGroup	Identifies the Swift app lifecycle entry point.
BluetoothManager.swift	@Published; service/characteristic UUIDs; BLE delegates	Documents observable telemetry and CoreBluetooth contract.
ContentView.swift	@State, @StateObject, Picker and validation guards	Documents UI controls and local validation behavior.

9.2 Required student-change annotation convention

For any verified student-authored modification, place paired markers around the edited source, preserve the original AI version in comments or repository history, and record the reason for the change. The following is an annotation **template**, not text found in the supplied files:

```
/* [HUMAN EDIT BEGIN - reason for change] */
/* AI original: <original implementation> */
/* Student revision: <specific change> */
/* [HUMAN EDIT END] */
```

AI-authored source code and generated documentation should be attributed as such. The E4235 library retains separate attribution to the University of Georgia course.

10. Implemented behavior and operational checks

The software contains the controls and data paths listed below. A source-level implementation should not be presented as independently measured hardware performance; physical measurements and signed demonstrations are distinct evidence.

Subsystem	Implemented behavior / test consideration
Motor and safety state	Manual START runs continuously until STOP or a safety stop. Bridge disable is commanded on stop, disconnect and shutdown.
Direction and speed	CW/CCW, Faster/Slower and RPM requests are handled on the Pi; over-range test commands use the explicit unlock state.
IR and encoder sensors	Independent workers publish IR RPM, encoder-derived RPM, encoder angle and validity/calibration status.
Zero and targets	RAM-based zero calibration, finite ZERO/ANGLE moves, direction probe and progress/timeout protections are implemented.
CSV evidence	The logging queue writes commanded steps, step angle, cumulative encoder angle and speed fields required for Test A and Test B analyses.
BLE interface	The iPhone receives ACK/ERR and rotating RPM, POS, STATE, LOG notifications; telemetry reflects Pi state.
Mode selection	Manual mode is implemented. Maintain control is not implemented in these files.
Power-on integration	The program initializes an inactive bridge when launched. The supplied files do not define an automatic startup service.

11. References and source traceability

Library descriptions, software behavior and identifiers are based on the project materials supplied for this document. The following references establish source ownership and the technical interfaces used.

Reference	Use in documentation
UGA ECSE 4235 PARMCO project guide, Fall 2026	Software documentation requirements, AI attribution, code flags and relevant test-data fields.
ECSE 4235 Raspberry Pi Library Documentation, revision 1.1.2 (12 February 2026)	Course library functions and attribution.
BlueZ GATT and LE Advertising D-Bus interfaces; bluez.readthedocs.io	GATT registration, advertising and notification model.
GNOME GLib/GIO references; docs.gtk.org	GDBus, GVariant, GMainLoop, GThread, GMutex and GAsyncQueue.
Apple Developer Documentation; developer.apple.com/documentation	SwiftUI, CoreBluetooth, Combine and Foundation frameworks.
Linux I2C userspace API; kernel.org	I2C_RDWR ioctl, repeated-start transactions and /dev/i2c-1 interface.
GCC documentation; gcc.gnu.org/onlinedocs	Compiler and linker flags used in the Raspberry Pi build.

11.1 Documented source identifiers

SHA-256 prefixes are identifiers for the source files reviewed. They are not Git commit IDs.

Project source file	SHA-256 prefix
motor_cp3.c	24fd7f7c132c
motor_cp3.h	479681b1d457
parmco_ble_cp3.c	578ca4c8cf0e
motorDriver.s	a99a4dc7f52a
PARMCOApp.swift	42dee6ec9001
BluetoothManager.swift	3e3701f00656
ContentView.swift	9dab035e1f23

11.2 Documentation provenance

Prepared through AI-assisted review of the seven supplied source files. The document records implemented code paths and explicit source configuration, but no compilation on the project Raspberry Pi, iOS device build, or new physical accuracy test was performed solely for this documentation release.